



Rapport de projet PC3R

Réalisation d'une application Web

AnimeTavern

Mai 2026

Sommaire

Table des matières

Sommaire.....	2
1. Sujet de l'application.....	3
2. Choix de l'API.....	3
3. Fonctionnalités de l'application.....	4
4. Cas d'utilisation classique de l'application.....	5
5. Base de données du serveur et mise à jour des données.....	6
User.....	6
Anime.....	6
UserAnime.....	7
Reviews.....	7
Mise à jour des données.....	8
6. Serveur (Backend).....	8
7. Client (Frontend).....	9
Plan du site.....	9
Navbar.....	9
Page d'accueil.....	10
Page d'un animé.....	10
Page de recherche.....	11
Profil utilisateur.....	12
Page d'inscription et de connexion.....	12
Requêtes entre Client et Serveur.....	13
Schéma de l'application.....	17
Commentaires sur l'utilisation de l'IA.....	17
Conclusion.....	18
Installation.....	18
Base de données	18
Serveur	20
Client.....	20
Démarrer l'application :.....	21

1. Sujet de l'application

AnimeTavern est une application dédiée au **suivi d'animés**. Les utilisateurs peuvent y enregistrer les animés qu'ils ont vus ou souhaitent voir et suivre leur progression en indiquant le nombre d'épisodes visionnés.

De plus, une fonctionnalité communautaire permet aux utilisateurs de publier des avis et d'attribuer des notes aux animés. Enfin, l'application propose une fonctionnalité de personnalisation permettant aux utilisateurs d'ajouter des animés à leurs favoris.

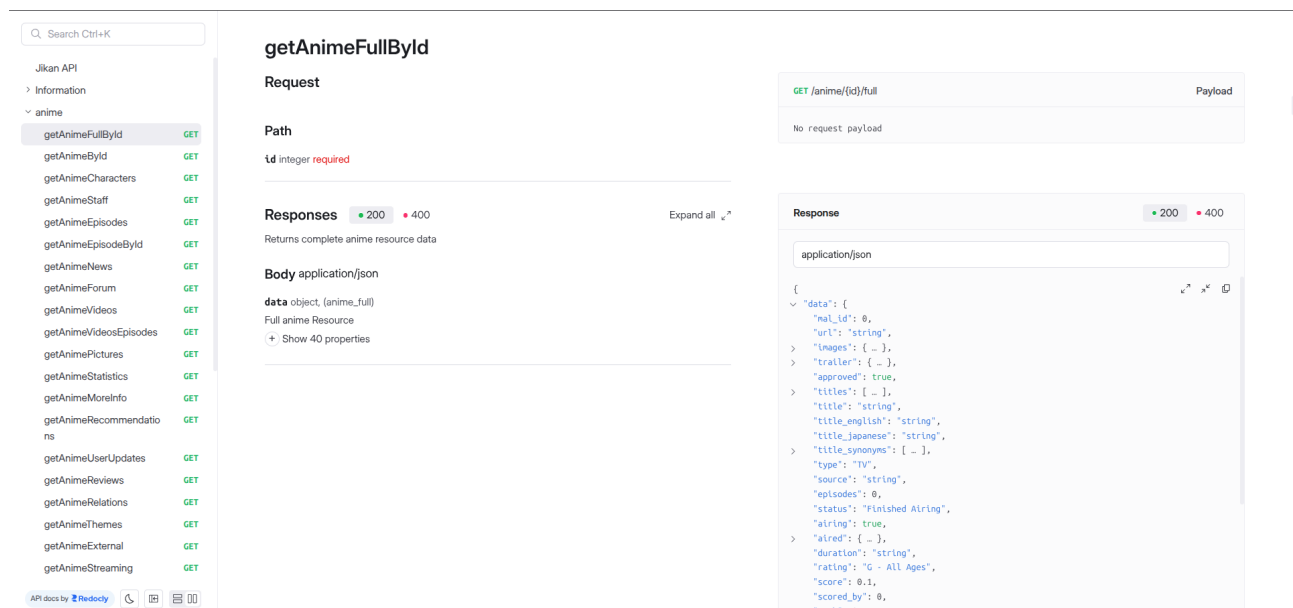
2. Choix de l'API

L'application utilise l'**API Jikan** <https://jikan.moe/> qui permet d'interagir avec la base de données de **MyAnimeList** <https://myanimelist.net/> qui est l'une des plus grandes plateformes consacrées aux mangas et aux animés.

Cette API est gratuite et limitée à **3 requêtes par seconde** et **60 requêtes par minute**. Afin de limiter le nombre de requêtes envoyées, l'application se concentre uniquement sur les animés.

Les données retournées par l'API sont au format JSON.

L'API propose de nombreuses routes permettant d'obtenir différentes informations sur un animé.

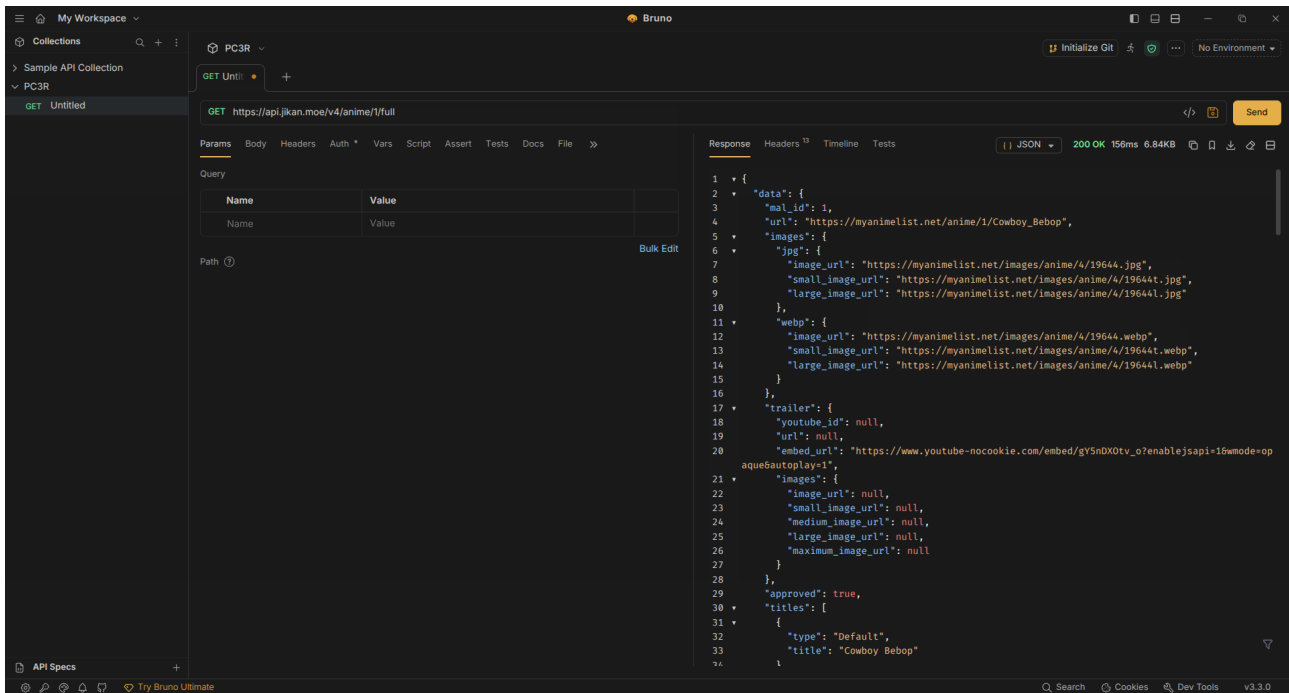


The screenshot displays the Jikan API documentation for the `getAnimeFullById` endpoint. On the left, a sidebar lists various API endpoints under the 'anime' category, with `getAnimeFullById` selected. The main panel shows the details for this endpoint, including the request method (GET), path (`/anime/{id}/full`), and response format (application/json). The response body is shown as a JSON object with fields like `mal_id`, `url`, `images`, `trailer`, `approved`, `titles`, `title_english`, `title_japanese`, `title_synonyms`, `type`, `source`, `episodes`, `status`, `airing`, `aired`, `duration`, `rating`, `score`, `scored_by`, and `rank`.

Toutefois, seule la requête qui permet d'obtenir les informations principales d'un animé est utilisé.

Voici l'exemple de requête qui sera utilisé par l'application :

<https://api.jikan.moe/v4/anime/1/full>



Cette requête retourne les informations de l'animé correspondant à l'identifiant 1.

3. Fonctionnalités de l'application

Les différentes fonctionnalités proposées par le site sont :

1. Inscription, connexion et déconnexion
2. Recherche d'un animé
3. Consultation de la page d'un animé
4. Ajout d'un animé à sa liste personnelle
5. Suppression d'un animé de sa liste personnelle
6. Modification du statut de visionnage d'un animé
7. Mise à jour du nombre d'épisodes visionnés d'un animé
8. Ajout et retrait d'un animé en favoris
9. Écriture d'un avis et notation d'un animé
10. Suppression d'un avis sur un animé
11. Consultation de la page utilisateur

Un utilisateur non connecté peut rechercher des animés et accéder à leurs pages, mais ne peut ni l'ajouter à sa liste ni à ses favoris.

À l'inverse, un utilisateur connecté peut effectuer ces actions uniquement pour son propre compte.

4. Cas d'utilisation classique de l'application

- Michel veut créer un compte sur l'application. Il se rend sur la page d'inscription, renseigne les informations demandées puis clique sur le bouton « **S'inscrire** ». Une fois l'inscription validée, il peut se connecter à son compte.
- Michel possède un compte et s'y connecte. Il recherche un animé depuis la page de recherche et trouve ce qu'il souhaite consulter. Il accède ensuite à la page de l'animé, lit ses informations puis le rajoute dans sa liste personnelle.
- Alice vient de terminer un épisode d'un animé présent dans sa liste personnelle. Elle se rend sur son profil et met à jour le nombre d'épisodes visionnés.
- Bob vient de terminer un animé qui est dans sa liste. Il se rend sur son profil et modifie le statut de l'animé à **TERMINÉ (FINISHED)** .
- Thomas souhaite se faire un avis sur un animé. Il accède à la page de l'animé et consulte les avis publiés. Un avis attire son attention et il souhaite en savoir plus sur son auteur, il se rend sur son profil et consulte sa liste d'animés.
- François possède un animé dans sa liste qu'il ne l'intéresse plus. Il se rend sur son profil et décide de supprimer l'animé de sa liste en appuyant sur le bouton « **Retirer** ».
- Thomas a terminé un animé et souhaite écrire un avis dessus. Il accède à la page de l'animé et clique sur le bouton « **Écrire un avis** » . Il est ensuite redirigé vers une page lui permettant d'écrire son avis puis appuie sur « **Envoyer** ». Son avis apparaît alors sur la page de l'animé ainsi que sur son profil.
- Frank a terminé un animé qu'il a particulièrement apprécié. Il se rend sur la page de l'animé pour l'ajouter dans ses favoris. Après avoir appuyé sur « **Ajouter à mes favoris** », l'animé apparaît dans la liste de ses favoris sur son profil.
- Antoine a publié un avis sur un animé qui ne correspond plus à son opinion actuelle. Il décide alors de retirer son avis. Pour cela, il rend sur son profil dans l'onglet « **Avis** » puis appuie sur le bouton « **Retirer** ».

5. Base de données du serveur et mise à jour des données

L'application web utilise une base de données SQL de type **PostgreSQL**.

Elle est composée de quatre tables principales :

User

Cette table contient les informations des utilisateurs de l'application. Elle permet notamment la gestion de l'inscription et de la connexion.

champs	id	email	username	password	created_at
type	INT	STRING	STRING	STRING	TIMESTAMP
description	Identifiant unique	Email	Pseudo	Mot de passe	Date d'inscription

Chaque utilisateur est identifié via un identifiant unique.

Le mot de passe des utilisateurs est **haché** avant d'être stocké dans la base de données afin de garantir une meilleure sécurité.

Anime

Cette table stocke les informations essentielles des animés récupérées depuis l'API externe.

champs	id	title	image	episodes
type	INT	STRING	STRING	INT
description	Identifiant de l'animé de l'API	Titre de l'animé	Url de l'image de l'animé de l'API	Nombre d'épisode de l'animé de l'API

Elle sert principalement de cache afin de limiter le nombre de requêtes envoyées à l'API Jikan.

Elle est notamment utilisée lors de l'affichage des informations d'un animé dans le profil des utilisateurs.

L'identifiant correspond à celui fourni par l'API.

Le champ image contient l'URL de l'image associée à l'animé.

UserAnime

Cette table gère les listes personnelles et les favoris des utilisateurs.

champs	id	user_id	anime_id	status	favorite	viewed_episodes
type	INT	INT	INT	STRING	BOOLEAN	INT
description	Identifiant unique pour la cohérence de la table	Identifiant de l'utilisateur	Identifiant de l'animé	Statut de visionnage de l'animé	Statut si l'animé est le favoris de l'utilisateur	Nombre de d'épisode visionné

Les champs **user_id** et **anime_id** sont des clés étrangères qui font référence aux tables **User** et **Anime**.

Le champ **status** peut uniquement prendre les valeurs suivantes : **PLANNING (à voir)**, **WATCHING (en cours de visionnage)**, **FINISHED (terminé)** .

Reviews

Cette table contient les avis publiés par les utilisateurs.

champs	id	user_id	anime_id	content	rating	created_at
type	INT	INT	INT	STRING	INT	TIMESTAMP
description	Identifiant unique	Identifiant de l'utilisateur	Identifiant de l'animé	Contenu de l'avis	Note donné par l'utilisateur	Date d'écriture de l'avis

Les champs **user_id** et **anime_id** sont des clés étrangères qui font référence aux tables **User** et **Anime**.

Un utilisateur ne peut publier qu'**un seul avis par animé**.

Mise à jour des données

Les données sont mises à jour à chaque **ajout ou modification** dans la liste d'un utilisateur, ainsi qu'à chaque publication ou suppression d'un avis.

6. Serveur (Backend)

Le serveur de l'application est développé en **Go** et suit une architecture de type **API REST**.

Son fonctionnement se repose sur des *handlers* répartis dans différents fichiers Go selon leur rôle. Chaque *handler* correspond à une **route** de l'API REST et permet de traiter les requêtes envoyées par les utilisateurs.

Les différents *handlers* de l'application sont les suivants :

animeHandler : gestion des fonctionnalités liées aux animés (page, avis, informations)

authHandler : gestion de l'authentification des utilisateurs

searchHandler : gestion de la recherche d'animés

userHandler : gestion des fonctionnalités liées aux utilisateurs (profil, ajout, suppression et mise à jour des listes)

Les handlers délèguent les opérations liées aux données à des *services* dédiés à l'accès et à la modification de la base de données.

Les requêtes sont filtrées par des *middlewares*, permettant d'autoriser ou de bloquer certaines requêtes selon des **conditions précises**.

Des fonctions utilitaires sont utilisées afin de vérifier la validité des requêtes effectuées par les utilisateurs, dans le but d'éviter les requêtes **incomplètes ou non autorisées**.

Les *models* sont des structures en Go qui correspondent à une entrée de la base de données, à une requête effectuée par l'utilisateur ou à une réponse renvoyée par le serveur.

Enfin, un fichier database est utilisé pour établir la connexion à la base de données. Celui-ci est ensuite utilisé par les différents *services* de l'application.

7. Client (Frontend)

Le client de l'application est développé en **React** <https://fr.react.dev/> et utilise **TailwindCSS** <https://tailwindcss.com/> pour l'interface utilisateur.

L'application suit une architecture **SPA (Single Page Application)**. La navigation entre les différents pages est gérée par **React Router** <https://reactrouter.com/> ce qui permet de changer de page sans recharger entièrement l'application.

Les pages sont regroupées dans un dossier *pages*, et utilisent des composants réutilisables situés dans le dossier *components*. Chaque page correspond à une fonctionnalité de l'application.

Les différentes pages de l'application sont :

- La page d'accueil
- La page de recherche
- La page d'un animé
- La page d'inscription et de connexion
- La page profil utilisateur

Les pages effectuent des **requêtes HTTP** vers l'API REST du serveur afin de récupérer ou modifier les données de l'application.

Enfin, l'application est développée en anglais afin de conserver une cohérence avec les données fournies par l'API Jikan.

Plan du site

Navbar

Une barre de navigation est présente en permanence afin de permettre une navigation fluide entre les différentes pages de l'application.

Deux versions de la navbar existent selon si l'utilisateur est connecté ou non :

Non connecté :



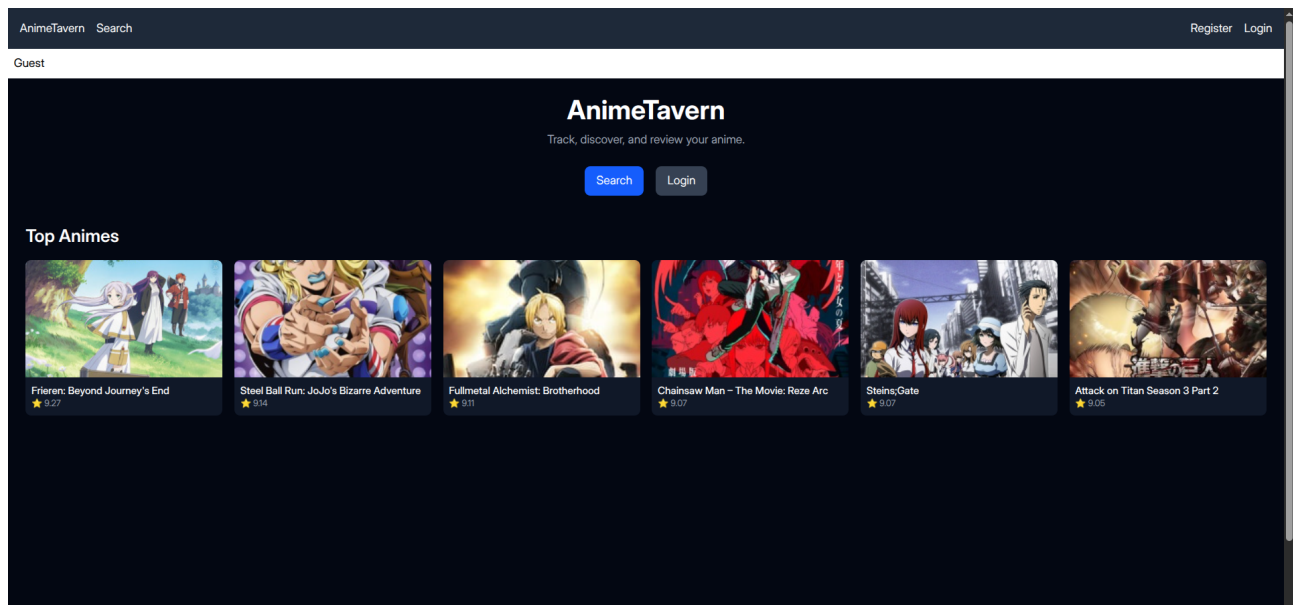
Connecté :

Elle permet d'accéder à:

- La page d'accueil
- La page de recherche
- La page d'inscription et de connexion
- Le profil utilisateur

Page d'accueil

Route : /



La page d'accueil permet à l'utilisateur de se repérer dans l'application et d'accéder aux principales fonctionnalités. Elle propose une sélection d'animés populaires.

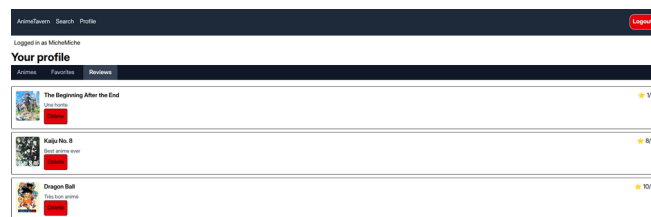
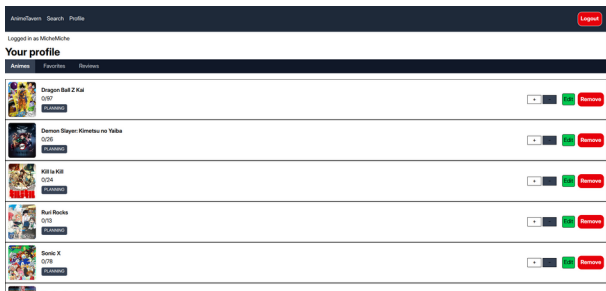
Page d'un animé

Route: /anime/:id

Cette page permet de rechercher des animés via une barre de recherche et d'afficher les résultats correspondants.

Profil utilisateur

Routes: /user/:id /user:id/favorites /user:id/reviews



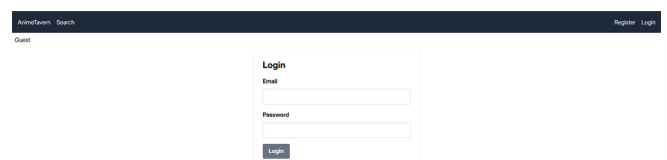
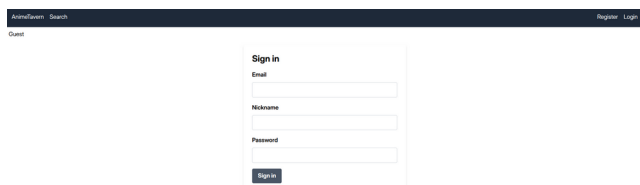
Cette page permet de consulter les informations d'un utilisateur.

Elle est divisée en plusieurs sections :

- liste des animés
- favoris
- avis publiés

Page d'inscription et de connexion

Routes : /register /login



Ces pages permettent de créer un compte ou de se connecter afin d'accéder aux fonctionnalités avancées de l'application.

Requêtes entre Client et Serveur

Le serveur est accessible en local à l'adresse : <http://localhost:8080>

Inscription :

Requête	Réponse
POST /register body : { email : « email donnée dans le formulaire » username : « pseudo donnée dans le formulaire » password : « mot de passe donnée dans le formulaire » }	201 Created

Connexion :

Requête	Réponse
POST /login body : { email : « email donnée dans le formulaire » password : « mot de passe donnée dans le formulaire » }	200 OK Création d'un cookie de session.

Déconnexion :

Requête	Réponse
POST /logout Credentials : cookie de session	200 OK Suppression du cookie de session { « Logged out » }

Recherche :

Requête	Réponse
GET /search?query=nom	200 OK { data : « données de l'API Jikan » }

Informations sur un animé :

Requête	Réponse
GET /anime/:id Credentials : cookie de session	200 OK { anime : « données de l'API Jikan » isInList : « pour savoir si il faut afficher le bouton pour ajouter l'animé » isFavorite : « pour savoir si il faut afficher le bouton pour mettre en favori l'animé » }

Ajout d'un animé à la liste :

Requête	Réponse
POST /user/anime Credentials : cookie de session body :{	201 Created { message : « Anime added »

anime_id : « id de l'animé » }	}
-----------------------------------	---

Suppression d'un animé de la liste :

Requête	Réponse
DELETE /user/anime/:id Credentials : cookie de session	204 No Content

Mise à jour du statut de visionnage d'un animé :

Requête	Réponse
PATCH user/anime/:id/status body{ status : « PLANNING, WATCHING OU FINISHED » }	200 OK { mal_id : « id de l'animé » status : « nouveau statut apres mise à jour » }

Les données sont renvoyés pour la mise à jour de l'interface client.

Mise à jour de visionnage d'un épisode d'un animé :

Requête	Réponse
PATCH /user/anime/:id/episodes Credentials : cookie de session body{ delta : « nombre d'épisode vu (-1 ou 1) » }	200 OK { mal_id : « id de l'animé » viewed_episodes : « episodes vu apres mise à jour » }

Les données sont renvoyés pour la mise à jour de l'interface client.

Ajout d'un animé en favoris :

Requête	Réponse
PATCH /user/anime/:id/favorite Credentials : cookie de session body{ favorite : true }	200 OK { favorite : « statut après mise a jour » }

Les données sont renvoyés pour la mise à jour de l'interface client.

Retrait d'un animé en favoris :

Requête	Réponse
PATCH /user/anime/:id/favorite Credentials : cookie de session body{ favorite : false }	200 OK { favorite : « statut après mise a jour » }

Les données sont renvoyées pour la mise à jour de l'interface client.

Ajout d'un avis d'un animé :

Requête	Réponse
POST /anime/:id/reviews Credentials : cookie de session body : { content : « contenu de l'avis » rating : « note » }	201 Created { message : « Review added » }

Suppression d'un avis d'un animé:

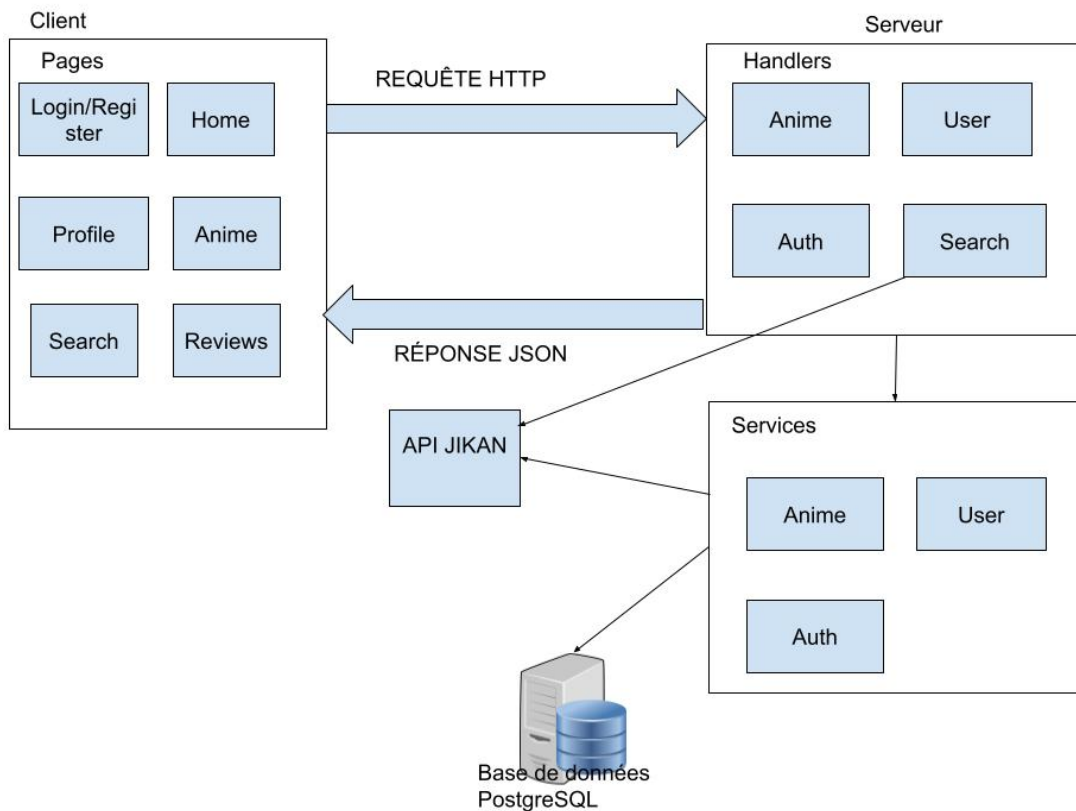
Requête	Réponse
DELETE /reviews/:id Credentials : cookie de session	200 OK { message : « Review deleted » }

Page utilisateur :

Requête	Réponse
GET /user/:id Credentials : cookie de session	200 OK { username : « pseudo de l'utilisateur » animes : « animés de l'utilisateur » favoriteAnimes : « animés favoris de l'utilisateur » }

	<pre> isUser : « pour savoir c'est le propriétaire du profil ou pas » } </pre>
--	--

Schéma de l'application



Commentaires sur l'utilisation de l'IA

L'intelligence artificielle ChatGPT a été utilisée durant le développement du projet. Elle a servi principalement d'aide pour certaines parties de l'authentification, pour des corrections ou améliorations de composants React et pour obtenir des explications sur certaines notions liées au framework React.

Conclusion

Ce projet m'a permis de développer une application Web complète reposant sur une architecture client/serveur avec React, Go et PostgreSQL.

Il m'a également donné l'occasion d'approfondir l'utilisation des API REST, la gestion d'une base de données relationnelle ainsi que le développement d'une interface web dynamique.

Enfin, ce projet a constitué une première expérience concrète avec le framework React et le déploiement d'une application web.

Installation

Pour déployer l'application localement, veuillez suivre les instructions suivantes :

Base de données

- Installer PostgreSQL sur votre machine

Linux : suivre les instructions d'installation selon la distribution en tant que sudo
<https://www.postgresql.org/download/linux/>

Windows : <https://www.postgresql.org/download/windows/>

- Demarrer postgresQL en tant que postgres : sudo -u postgres psql (Linux) ou

psql -U postgres -h localhost -p 5432 (Windows)

- Créer la base de données de l'application : CREATE DATABASE animetavern_db;

- Verifier que la base de données a bien été crée : \l

- Créer l'utilisateur myuser : CREATE USER myuser WITH PASSWORD 'mypassword';

- Donner à myuser les permissions sur la base de données : GRANT ALL PRIVILEGES ON DATABASE animetavern_db TO myuser;

- Quitter PostgreSQL : \q

Se reconnecter avec myuser :

Linux : psql -U myuser -d animetavern_db

Windows:psql -U myuser -h localhost -p 5432 -d animetavern_db

- Créer les tables de l'application :

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    email TEXT NOT NULL UNIQUE,  
    username TEXT NOT NULL UNIQUE,  
    password TEXT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE anime (  
    id INTEGER PRIMARY KEY,  
    title VARCHAR(255),  
    image TEXT,  
    episodes INTEGER DEFAULT 0  
);
```

```
CREATE TABLE user_anime (  
    id SERIAL PRIMARY KEY,  
  
    user_id INTEGER NOT NULL,  
    anime_id INTEGER NOT NULL,  
  
    status VARCHAR(50) DEFAULT 'PLANNING',  
    note INTEGER,  
    favorite BOOLEAN DEFAULT FALSE,  
    viewed_episodes INTEGER DEFAULT 0,  
  
    CONSTRAINT unique_user_anime UNIQUE (user_id, anime_id),  
  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
```

```
FOREIGN KEY (anime_id) REFERENCES anime(id) ON DELETE CASCADE  
);
```

```
CREATE TABLE reviews (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL,  
  anime_id INTEGER NOT NULL,  
  content TEXT NOT NULL,  
  rating INTEGER,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (anime_id) REFERENCES anime(id) ON DELETE CASCADE,  
  UNIQUE (user_id, anime_id)  
);
```

Serveur

Installer Go : <https://go.dev/doc/install>

Cloner le repo : https://github.com/Slashinkun/animetavern_backend

A l'aide d'un terminal , se diriger dans le répertoire du serveur : `cd /server`

Installer les dépendances : `go mod tidy`

Créer le fichier .env dans le répertoire du serveur avec ces paramètres :

DB_HOST=localhost

DB_PORT=5432

DB_USER=myuser

DB_PASSWORD=mypassword

DB_NAME=animetavern_db

DB_SSLMODE=disable

Vérifier que le serveur démarre : `go run main.go`

Client

Installer NodeJS : <https://nodejs.org/fr>

Cloner le repo : https://github.com/Slashinkun/animetavern_frontend

A l'aide d'un terminal, se situer dans le répertoire du client

Installer les dépendances : `npm install`

Vérifier que le client marche : `npm run dev`

Démarrer l'application :

Démarrer le serveur et le client dans 2 terminaux séparés